

FortiFlow

Version : août 2026 · Commit : 893abb8 · Tests : 49/49 [OK]

À propos

FortiFlow est un outil d'analyse de logs trafic FortiGate / FortiAnalyzer pour les prestations de segmentation réseau. Ce guide couvre l'ensemble du cycle de vie : analyse de code, déploiement Docker (VPS et Portainer), configuration HTTPS et maintenance.

Ce document couvre l'ensemble du cycle de vie de FortiFlow : analyse du code, déploiement Docker (VPS et Portainer), configuration HTTPS et maintenance.

1. Présentation

FortiFlow est un outil d'analyse de logs trafic FortiGate / FortiAnalyzer pour les prestations de segmentation réseau. Il importe les logs trafic, conserve leur contexte FortiGate/VDOM, construit les matrices de flux, rapproche les observations de la configuration et du routage, puis prépare une CLI FortiGate destinée à être revue par un ingénieur.

- Application web : Node.js + Express dans `app/web/`
- CLI Python historique : `fortiflow.py` (sans dépendance externe)
- Docker : image `node:22-alpine`, utilisateur non-root `fortiflow`

2. Synthèse de la revue de code

Revue réalisée le 3 août 2026 par l'équipe Da Vinci (Socrate, Archimède, Ada).

2.1 Points forts

Domaine	Évaluation
Modèle fail-closed	Exemplaire — 15 invariants de sécurité documentés et tracés jusqu'au code
Parseur multi-format	KV, CSV, CSV-KV, ZIP, XLSX — détection automatique du format
Ségrégation VDOM/équipement	Aucune fusion silencieuse entre scopes différents
Preflight + deploy-safety	Double vérification avant toute génération CLI
Tests	49 tests (1023 lignes de tests de sécurité) — 0 échec
Limites configurables	Tous les points d'entrée ont des garde-fous (taille upload, mémoire, workers)
Séparation des privilèges	Container non-root via <code>`su-exec`</code> , bind <code>`127.0.0.1`</code> uniquement

2.2 Corrections appliquées

#	Correction	Impact
1	<code>`node:20-alpine`</code> → <code>`node:22-alpine`</code>	47 CVE corrigées

#	Correction	Impact
2	Ajout `healthcheck` Docker	Monitoring de santé du conteneur
3	`security_opt: no-new-privileges` + `tmpfs /tmp`	Durcissement sécurité
4	Rate limiting token-bucket sur `/api/upload` + `/api/admin`	Protection brute-force
5	Graceful shutdown (`SIGTERM`/`SIGINT`)	Arrêt propre avec fermeture du pool
6	Module partagé `lib/constants.js`	Déduplication de `ALLOW_ACTIONS` / `DENY_ACTIONS`
7	Suppression `ecosystem.config.js`	PM2 non utilisé, fichier mort
8	CI : build Docker + scan Trivy + push GHCR	Pipeline de sécurité complet
9	`.env.example` + config Nginx	Documentation des variables et reverse proxy

2.3 Points restants (non bloquants)

Point	Priorité	Contexte
Pas d'authentification	—	L'outil est en local uniquement (pas d'exposition WAN)
Sessions in-memory	Medium	Perdus au redémarrage — acceptable pour l'usage
`ip2int` / `isPrivate` dupliqués	Low	analyzer.js + forticonfig.js — refactoring futur
Tests d'intégration (supertest)	Low	Couverture unitaire OK, pas d'intégration HTTP

3. Architecture

```

app/web/
├─ server.js # Express + WebSocket + SSE (2079 lignes)
├─ entrypoint.sh # chown → su-exec fortiflow
├─ Dockerfile # node:22-alpine, non-root
├─ lib/
│  ├─ parser.js # Parseur logs (KV, CSV, ZIP, XLSX)
│  ├─ analyzer.js # Analyse de flux, matrices, politiques
│  ├─ forticonfig.js # Parseur config FortiGate
│  ├─ deploy-safety.js # Certification avant déploiement
│  ├─ coverage.js # Couverture de politiques existantes
│  ├─ analysis-pool.js # Pool de workers (hors thread HTTP)
│  ├─ constants.js # Constantes partagées (ALLOW_ACTIONS, etc.)
│  ├─ ports.js # Noms de ports IANA
│  └─ store.js # Sessions in-memory
├─ public/
│  ├─ index.html # Interface principale
│  ├─ app.js # Logique frontend
│  ├─ segmentation-plan.js # Plan de segmentation
│  └─ style.css
├─ admin.html # Page d'administration
├─ test/ # 49 tests (sécurité, parsing, analyse)
└─ package.json # express, ws, multer, xlsx, exceljs, unzipper

```

4. Maj Docker

FortiFlow est déjà déployé sur le VPS. Les commandes ci-dessous suffisent pour la maintenance courante.

4.1 Mise à jour

```

cd ~/workspace/FortiFlow
git pull
docker compose up --build -d

```

4.2 Vérification

```

docker compose ps
# Doit afficher "(healthy)"

docker compose logs -f
# Doit afficher : FortiFlow → http://localhost:3737 ...

```

5. Déploiement Portainer (serveur interne)

Le fichier `docker-compose.portainer.yml` est prêt pour un déploiement sans build via Portainer.

5.1 Prérequis

- VM interne avec Docker et Portainer Community Edition
- Accès SSH à la VM pour placer les certificats
- Règles firewall limitant l'accès au réseau interne autorisé

5.2 Fichiers nécessaires

Fichier	Origine	Utilisation
`fortiflow.tar`	Buildé depuis le dépôt	Import image dans Portainer

Fichier	Origine	Utilisation
<code>`docker-compose.portainer.yml`</code>	Dépôt Git	Stack Portainer

5.3 Construction de l'image

```
cd ~/workspace/FortiFlow/app/web
docker build -t fortiflow-fortiflow:latest .
docker save fortiflow-fortiflow:latest -o ~/fortiflow.tar

# Vérifier
ls -lh ~/fortiflow.tar
sha256sum ~/fortiflow.tar > ~/fortiflow.tar.sha256

# Transférer vers le poste qui ouvre Portainer
scp ~/fortiflow.tar user@POSTE:/tmp/
```

5.4 Import dans Portainer

1. Ouvrir Portainer → sélectionner l'environnement Docker cible
2. Menu Images → Import
3. Sélectionner `fortiflow.tar` → confirmer
4. Vérifier que `fortiflow-fortiflow:latest` apparaît

5.5 Création de la Stack

1. Menu Stacks → Add stack
2. Nom : `fortiflow`
3. Méthode : Upload → sélectionner `docker-compose.portainer.yml`
4. Deploy the stack

5.6 Mise à jour

Deux méthodes sont disponibles pour mettre à jour FortiFlow sans refaire l'installation complète de la stack.

Utiliser cette méthode quand le serveur interne n'a pas accès à internet ou à GHCR.

```
# Étape 1 : Reconstruire l'image localement
cd ~/workspace/FortiFlow/app/web
docker build -t fortiflow-fortiflow:latest .

# Étape 2 : Exporter et vérifier l'image
docker save fortiflow-fortiflow:latest -o ~/fortiflow.tar
ls -lh ~/fortiflow.tar
sha256sum ~/fortiflow.tar > ~/fortiflow.tar.sha256

# Étape 3 : Transférer vers le serveur interne
scp ~/fortiflow.tar user@SERVEUR_INTERNE:/tmp/
```

Sur le serveur interne :

```
# Étape 4 : Charger la nouvelle image
docker load -i /tmp/fortiflow.tar

# Étape 5 : Redéployer dans Portainer
# → Ouvrir Portainer → Stacks → fortiflow → Stop → Start
# L'image fortiflow-fortiflow:latest sera automatiquement utilisée
```

Quand le serveur interne dispose d'un accès à internet, l'image est publiée automatiquement sur GitHub Container Registry par le pipeline CI/CD. Il suffit de tirer la nouvelle image et de redéployer.

```
# Sur le serveur interne
docker pull ghcr.io/tetrax/fortiflow:latest
docker tag ghcr.io/tetrax/fortiflow:latest fortiflow-fortiflow:latest

# Puis dans Portainer : Stop → Start la stack fortiflow
```

Note

Note : Le tag :latest sur GHCR est mis à jour automatiquement à chaque push sur la branche main (voir section 11).

6. HTTPS

6.1 Structure attendue

Sur l'hôte, créer le répertoire `/etc/ssl/fortiflow/` :

```
/etc/ssl/fortiflow/
├─ privkey.pem # Clé privée (droits 600)
└─ fullchain.pem # Certificat + chaîne intermédiaire
```

6.2 Option A -- Let's Encrypt (VPS)

```
sudo mkdir -p /etc/ssl/fortiflow
sudo cp /etc/letsencrypt/live/<DOMAINE>/privkey.pem /etc/ssl/fortiflow/privkey.pem
sudo cp /etc/letsencrypt/live/<DOMAINE>/fullchain.pem /etc/ssl/fortiflow/fullchain.pem
sudo chmod 600 /etc/ssl/fortiflow/*.pem

# Ajouter un hook de renouvellement automatique
cat << 'EOF' | sudo tee /etc/letsencrypt/renewal-hooks/deploy/fortiflow.sh
#!/bin/bash
cp /etc/letsencrypt/live/$RENEWED_DOMAIN/privkey.pem /etc/ssl/fortiflow/privkey.pem
cp /etc/letsencrypt/live/$RENEWED_DOMAIN/fullchain.pem /etc/ssl/fortiflow/fullchain.pem
chmod 600 /etc/ssl/fortiflow/*.pem
docker restart fortiflow
EOF
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/fortiflow.sh

# Redémarrer le conteneur pour activer HTTPS
docker restart fortiflow
```

6.3 Option B -- Certificat wildcard interne (PKI entreprise)

```
sudo mkdir -p /etc/ssl/fortiflow
# Récupérer le certificat depuis la PKI (AD CS, EJBCA, etc.)
sudo cp wildcard.key /etc/ssl/fortiflow/privkey.pem
sudo cp wildcard.crt /etc/ssl/fortiflow/fullchain.pem
sudo chmod 600 /etc/ssl/fortiflow/*.pem
docker restart fortiflow
```

6.4 Option C -- Self-signed (test/local)

```
sudo mkdir -p /etc/ssl/fortiflow
openssl req -x509 -newkey rsa:4096 -days 3650 -nodes \
-keyout /etc/ssl/fortiflow/privkey.pem \
-out /etc/ssl/fortiflow/fullchain.pem \
-subj "/CN=fortiflow" \
-addext "subjectAltName=IP:<IP_SERVEUR>,DNS:fortiflow.local"
docker restart fortiflow
```

Le serveur détecte automatiquement la présence des certificats et bascule en HTTPS. S'ils sont absents, il reste en HTTP.

7. Reverse proxy Nginx

Une configuration Nginx est fournie dans `infra/nginx/fortiflow.conf`.

```
sudo cp infra/nginx/fortiflow.conf /etc/nginx/sites-available/
sudo ln -s /etc/nginx/sites-available/fortiflow.conf /etc/nginx/sites-enabled/
# Adapter server_name et chemins de certificats
sudo vim /etc/nginx/sites-available/fortiflow.conf
sudo nginx -t && sudo systemctl reload nginx
```

Points clés de la configuration :

- Proxy WebSocket (Upgrade, Connection) pour `/ws/progress`
- `client_max_body_size 2048m` pour les exports FAZ volumineux
- Timeouts élevés (3600s) pour les analyses longues
- Renvoi vers `127.0.0.1:13737` (port interne FortiFlow)

Note

Sans reverse proxy : Si aucun reverse proxy (Nginx, Apache, Traefik) n'est présent sur le serveur, vous pouvez exposer directement le port 443 dans `docker-compose.yml`. Remplacez `127.0.0.1:13737:3737` par `443:3737` dans la section `ports`. FortiFlow détecte automatiquement les certificats dans `/etc/ssl/fortiflow/` et servira le HTTPS directement, sans intermédiaire. Assurez-vous que le port 443 est ouvert dans le firewall et qu'aucun autre service ne l'utilise.

8. Sauvegarde et restauration

Backup

```
cd ~/workspace/FortiFlow
tar -czf fortiflow-backup-$(date +%Y%m%d).tar.gz data/
```

Restauration

```
cd ~/workspace/FortiFlow
tar -xzf fortiflow-backup-YYYYMMDD.tar.gz
docker compose up -d # ou redeploy via Portainer
```

9. Variables d'environnement

Variable	Défaut	Description
`PORT`	`3737`	Port d'écoute du serveur
`DOMAIN`	`devval.com`	Nom de domaine (utilisé pour les logs)
`MAX_UPLOAD_SIZE_MB`	`2048`	Taille max d'un export FAZ (Mo)
`MAX_DECOMPRESSED_SIZE_MB`	`4096`	Taille max après décompression (Mo)
`MAX_ARCHIVE_ENTRIES`	`100`	Nombre max d'entrées dans une archive
`MAX_XLSX_SIZE_MB`	`100`	Taille max d'un fichier XLSX (Mo)
`MAX_WORKSPACE_UNCOMPRESSED_MB`	`1024`	Taille max d'un workspace (Mo)
`MAX_SESSION_DEDUPE_KEYS`	`2000000`	Identifiants de session max pour déduplication
`MAX_ANALYSIS_WORKERS`	`1`	Nombre de workers d'analyse
`MAX_ANALYSIS_QUEUE`	`3`	Taille de la file d'attente d'analyse
`ANALYSIS_WORKER_MEMORY_MB`	`0`	Plafond mémoire par worker (0 = défaut Node)
`SSL_KEY`	*(vide)*	Chemin vers la clé privée dans le conteneur (`/certs/privkey.pem`)
`SSL_CERT`	*(vide)*	Chemin vers le certificat dans le conteneur (`/certs/fullchain.pem`)
`CUSTOM_SSL_DIR`	`/etc/ssl/fortiflow`	Répertoire hôte des certificats

10. Résolution de problèmes

Symptôme	Cause probable	Solution
Container `unhealthy`	Node.js ne répond pas	`docker logs fortiflow`
HTTP mais pas HTTPS	Certificats absents	Vérifier `/etc/ssl/fortiflow/`
`Permission denied` sur `/certs`	Droits incorrects	`chmod 644` sur les .pem

Symptôme	Cause probable	Solution
Portainer : `no such image`	Image non importée	`docker load -i fortiflow.tar`
Nginx : 502 Bad Gateway	FortiFlow ne tourne pas	`docker ps`, vérifier les logs
Erreur `ANALYSIS_CANCELLED`	Analyse interrompue	Relancer l'upload
Erreur `DECOMPRESSED_SIZE_LIMIT`	Archive trop volumineuse	Augmenter `MAX_DECOMPRESSED_SIZE_MB`

11. CI/CD

Le workflow GitHub Actions ([.github/workflows/security-tests.yml](#)) exécute :

1. node-tests : vérification syntaxe + 49 tests
2. docker-build : build image → démarrage → healthcheck → tests dans conteneur
3. security-scan : Trivy (sortie en erreur si CRITICAL ou HIGH)
4. publish-ghcr : push vers `ghcr.io/tetrax/fortiflow:latest` (sur main uniquement)